

Mid-Project Report - ProGym

GitHub Repository:

<https://github.com/kmc4699/ProGym>
[m](#)

Task 1: Problem Definition and Approval

The problem

Picture a Monday morning class at a small gym. The instructor's ready, and so are twelve members, even though the room only fits eight. Two of them booked the same slot without meaning to. One person's membership expired last week; nobody noticed. That's the reality when a gym runs its bookings on paper sign-up sheets, whiteboards, and shared spreadsheets. Classes get overbooked past what the space can safely hold. Members are allowed to book on expired memberships. And when a booking fails, nobody's told why. On the surface it looks like a small workflow annoyance, but underneath it's a software quality problem: the process has almost no validation, no reliable rules, and no clear feedback. We're calling our project ProGym. It's a small gym management system with four modules that connect: membership (with expiry and renewal), class booking, check-in for attendance, and a reporting dashboard.

Stakeholders

- Members: want a booking they can trust and a spot when they show up.
- Front-desk staff: need something that doesn't confuse them during the mid-morning rush.
- Instructors: rely on accurate class lists and capacity being respected.
- Gym owner: cares about member data being safe and classes filled sensibly.

Key software quality issues

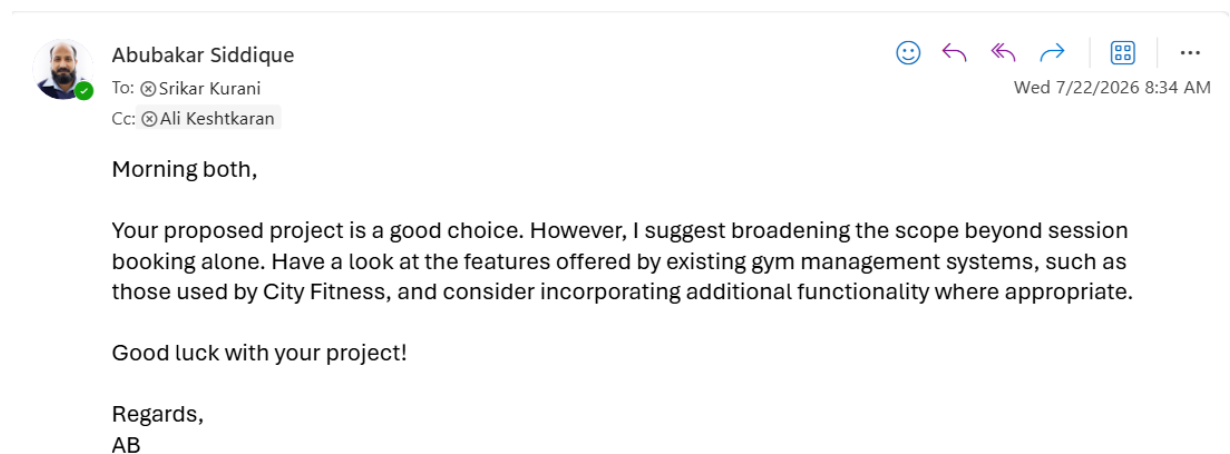
- Reliability: a class must never be booked past its capacity, no matter what.
- Correctness and validation: bad input (expired members, past-date bookings) must be caught early, not silently accepted.
- Usability: every action needs a clear, plain-language result so staff aren't left guessing.
- Data integrity: the system shouldn't be able to end up in impossible states (like negative slot counts).
- Maintainability: rules should live in one place so they're easy to change without breaking everything.

Feasibility

It's realistic for one semester. The core of each module is small, and we've kept the scope tight: no real payments, no SMS, no online booking, no database. Those are the things we deliberately left out so we can finish and test what we build.

Approval

We emailed our lecturer with the proposal early in the semester and got approval. The feedback was: Good choice but broaden the scope beyond just session booking - look at what real gym management systems (like the ones at City Fitness) do. We took that on board and expanded it into the four connected modules we're building now. The approval email is saved as evidence.



Task 2: Requirements and Quality Analysis

We started by turning the problem into concrete requirements. Sixteen functional (FR1-FR16) and eight non-functional (NFR1-NFR8), grouped by module.

Functional requirements (by module)

- Membership (FR1-FR3): register a member with ID, name, and expiry date; reject blank ID or name; treat a membership as active or expired based on today's date.
- Booking (FR4-FR11): book an active member into a class with a free slot; reject full classes and expired members; reject past-date bookings; stop double-booking clashes; reduce free slots on booking and return them on cancel; always give a clear message.
- Check-in / Attendance (FR12-FR13): mark a booked member as attended or no-show; only allow check-in for someone who has a confirmed booking.
- Reporting (FR14-FR16): show active vs expired member counts, class utilisation (booked out of capacity), and attendance summaries.

Non-functional requirements

The main NFRs are reliability (capacity never exceeded - NFR1), usability (every result carries a plain-language message - NFR2), performance (single-user actions under 1 s - NFR3), maintainability (rules live in one place - NFR4), data integrity (invalid states impossible to construct - NFR5), testability (every rule has at least one automated test - NFR6), portability (runs on any .NET 10 platform - NFR7), and accessibility (messages don't rely on colour alone - NFR8).

Analysis against the six quality dimensions

- Clarity - most requirements use exact conditions (dates, slot counts, active/expired), so they're hard to misread. A few first drafts said things like "the system should be user-friendly"; we rewrote those.
- Completeness - we cover success paths and error cases (full, expired, past date, clash). There are gaps (payments, reminders, login) that we've listed as out of scope rather than pretending they don't exist.
- Consistency - the rules line up. The capacity rule in FR5/FR9 matches what Reporting shows in FR15. The expiry rule in FR3 is the same one used in FR6.

- Correctness - the rules match how gyms work. A class has a hard capacity; memberships really do expire.
- Feasibility - all of these are achievable in C# with MSTest inside the semester.
- Testability - each FR maps to a clear pass/fail assertion.

A rewrite for testability (Before & After)

- Before: "The system should prevent overbooking."
- After: "FR5: Given a class with 0 free slots, when a member tries to book, the system rejects the booking and the class slot count stays at 0."

It is now unambiguous and directly testable, unlike the original one.

Acceptance criteria (Given/When/Then)

We wrote these using the Given/When/Then format popularised by North (2006) as part of Behaviour-Driven Development.

- Successful booking - Given an active member and a class with a free slot, when they book, then it succeeds, one slot is consumed, and the message names the member and class.
- Full class - Given a class with zero free slots, when a member tries to book, then it's rejected, the count stays the same, and the message says, "fully booked".
- Expired membership - Given a member whose expiry date has passed, when they try to book, then it's rejected with the reason "membership expired".

Relevant quality attributes (ISO/IEC 25010)

Reliability, functional suitability, usability, security and data integrity, maintainability, performance efficiency, accessibility, and portability. These come from the ISO/IEC 25010 (2011) quality model. Those are the eight we're targeting; the others (like context coverage) are less relevant to a small gym prototype.

Task 3: Proposed Solution and Initial Prototype

How the solution addresses the problem

ProGym directly targets the four quality problems from Task 1. Overbooking is prevented by the class enforcing its own capacity. Expired memberships can't book. Every booking attempt returns a clear message with a reason. Each module keeps its rules in one place, so validation isn't scattered around.

What we've built

- **Membership:** holds ID, name, expiry. Validates blank ID/name in the constructor. `IsActive()` tells you if it's still valid, `Renew(newDate)` extends the expiry (and refuses a date that isn't later than the current one), and `DaysUntilExpiry()` returns how many days are left.
- **FitnessClass:** a class/session with name, start time, capacity, and booked count. `ReserveSlot()` throws if full. `ReleaseSlot()` gives a place back. `BookedCount` is private set, so nothing outside can secretly change it.
- **Booking:** a record linking a member to a class, with a cancelled state.
- **BookingResult:** the outcome of a booking or cancel - success, message, and (if successful) the booking. This is what powers the "clear message" requirement.
- **BookingService:** the brain of booking. Checks the member is active, checks the class has room, reserves a slot, returns a `BookingResult`. Also, `CancelBooking(booking)`, which releases the slot back.
- **CheckIn:** records a member checking in. Refuses if the membership is expired.
- **ReportingService:** reads the in-memory collections and returns summaries - active vs expired members, class utilisation, and total check-ins.

Interface

The UI is a Blazor Server web app (`GymManagement.Web`, .NET 10). We picked Blazor over a desktop GUI because it's platform-independent, easier to demo (just a browser), and lets us keep all the code in C#. It has four pages driven by a shared `GymDataStore` that holds the in-memory lists:

- Dashboard (home): shows the summary cards from ReportingService: active vs expired members, total check-ins, and a class utilisation table.
- Members: register a member, renew, check-in, and see the list with days-until-expiry.
- Classes: add a class (validation errors are shown clearly) and see all classes with booked/capacity/available.
- Bookings: book a member into a class, cancel a booking, and see the result message on every attempt.

Design decisions that show quality awareness

- Encapsulation: BookedCount, ExpiryDate, and IsCancelled all use a private set. The only way to change them is through methods that enforce the rules, so outside code can't corrupt state.
- Constructor validation: every domain class rejects invalid input in its constructor. An invalid object can't be created.
- Result objects instead of throwing for expected outcomes: a full class or expired member isn't a crash, it's a normal outcome, so we return a BookingResult with a clear message instead of an exception.
- Slot rules kept inside FitnessClass: any code that wants to book must go through ReserveSlot(), which has the "if full, throw" check. The invariant "booked count never exceeds capacity" is protected by the type itself.
- In-memory storage on purpose: no database in the prototype. Keeping data in memory means unit tests don't touch external systems, which is easier to trust and much faster. GymDataStore, BookingService, and ReportingService are registered as singletons, so every page sees the same data.

Task 4: AI-Assisted Development Using GitHub Copilot

We treated AI assistance the way the course expects - useful for ideas but never accepted blindly. Every suggestion had to be reviewed, understood, tested, and often modified before it went in. Below are honest examples from building ProGym.

Where we used AI

Mostly for boilerplate - MSTest scaffolding, domain class shells, Blazor page structure, and DI registration lines. It sped up the routine parts and let us spend more time on design decisions like encapsulation and how our error results should look. AI was also useful for explaining C# compiler errors that referenced unfamiliar concepts.

One suggestion we kept (and why)

The BookingResult result-object pattern came from an AI suggestion. It carries a Success flag, a plain-language message, and (on success) the created Booking itself. We kept it because it fit our "every attempt returns a clear message" requirement (FR11 / NFR2) perfectly. We reviewed the shape, wrote the class ourselves, and added tests around the message content before we considered it done.

One suggestion we modified

After we added a past-expiry guard to Membership's constructor, five tests that constructed expired members with past dates suddenly broke. An AI suggestion proposed introducing an IClock abstraction and a FakeClock test double so tests could "advance time" rather than pass a past date. We took the pattern but re-wrote each test carefully to fit our style. The result: expired memberships can now be tested cleanly without violating the constructor rule, and all 30 tests pass on master again.

The broken test helper before and after:

```
// before - broke when the past-expiry guard was added
private static Membership ExpiredMember() =>
    new Membership("M002", "John Chen", DateTime.Today.AddDays(-5));

// after - construct active, then advance the fake clock
private static Membership ExpiredMember()
{
    var clock = new FakeClock { Today = DateTime.Today };
    var m = new Membership("M002", "John Chen", DateTime.Today.AddDays(5), clock);
    clock.Today = DateTime.Today.AddDays(10);
    return m;
}
```

One suggestion we rejected

When we built the Bookings page, an AI suggestion wrapped the `BookingService.BookClass(...)` call in a `try/catch`, treating a full class or an expired member as an exception. We rejected it. Our `BookingService` deliberately returns a `BookingResult` for these expected outcomes instead of throwing, so the `try/catch` added noise - the page just checks `result.Success` and shows the message. In our design, exceptions mean genuine misuse (like calling `ReserveSlot()` on a full class), not normal business rejections.

The two options side by side:

```
// AI suggestion (rejected)
try {
    var r = BookingSvc.BookClass(m, c);
    message = r.Message;
} catch (Exception ex) {
    message = ex.Message;
    isError = true;
}

// what we wrote instead
var r = BookingSvc.BookClass(m, c);
message = r.Message;
isError = !r.Success;
```

Reflection

The best thing about AI-assisted coding is momentum - we're not stuck typing boilerplate. The worst is that a suggestion that looks confident can still be wrong in a way that only shows up when a test fails, or worse, when it doesn't. The habit that saved us most often was running the tests immediately after accepting any AI-generated code. If the tests weren't green, we knew before it hit the repo. That fits neatly with our team rule of "code must pass all tests before commit".

Task 5: Initial Test Strategy and Test Planning

Purpose

This strategy sets out how we'll test ProGym. The goal is to prove that the rules we've written (membership, booking, check-in, dashboard) work - and keep working as we add features.

Scope

- In scope: unit testing of every domain class in GymManagement; component testing of BookingService and ReportingService (they exercise multiple classes together); black-box validation testing on constructors and business rules; and end-to-end manual testing through the Blazor web app.
- Out of scope for the mid-project: payments, real SMS or email, native mobile apps, integration with third-party gym systems, and a database. Automated UI testing is limited to running the app ourselves and checking the pages.

Test levels and types

- Unit tests - single classes in isolation (FitnessClass, Membership, CheckIn, Booking).
- Component tests - BookingService.BookClass calls into FitnessClass and Membership, so those tests are component tests. We call them out because the boundary matters.
- Validation testing - rejecting invalid input (blank IDs, negative capacity, expired members, full classes, null arguments).
- Boundary-value tests - slot count 0 vs 1, expiry today vs yesterday vs tomorrow.
- Manual end-to-end testing - driving the Blazor pages ourselves to confirm the pages, the shared data store, and the domain services all work together correctly.
- Regression testing - the whole suite runs on every commit; nothing merges to master with unless all tests are passing.

Approach

We wrote tests alongside every feature. Each functional requirement has at least one test. We use MSTest's arrange-act-assert pattern, favour Assert.Throws<T> for negative cases and keep tests independent (no shared mutable state).

Non-functional testing

Some NFRs are naturally covered by unit tests - reliability (NFR1) is proven by "reserve when full throws"; data integrity (NFR5) by the constructor validation tests. Others (performance NFR3, accessibility NFR8) will be manually checked once the pages are all complete.

Environment and tools

- Visual Studio 2026 with MSTest Test Explorer.
- .NET 10 SDK, MSTest 4.3.3, Blazor Server for the web UI.
- dotnet test from the command line for build-time checks.
- Git and GitHub for source control and test evidence (test-passing commits are the evidence trail).
- GitHub Copilot for test-idea generation (reviewed and modified, see Task 4). These tools are documented in Microsoft's official MSTest and Blazor guides (Microsoft, n.d.-a, n.d.-b) and GitHub's Copilot documentation (GitHub, n.d.).

Defect management, entry/exit criteria, risks

Any defect is logged with an ID, description, severity, and status. It's only marked "Fixed" once a test confirms the corrected behaviour. Entry criteria: code compiles cleanly and every new feature has at least one requirement ID and one test. Exit criteria: all unit tests pass on master, no open high-severity defects, and every in-scope FR has at least one test. The main risks are one module breaking another (mitigated by running all tests before every commit), over-relying on AI test suggestions (mitigated by the review process in Task 4), and uneven contribution (mitigated by both members committing directly to master under their own accounts).

Task 6: Initial Test Cases and Requirements Traceability

The tables below list a selection of our test cases (30 pass on origin/master at time of writing) and then a traceability matrix linking every FR to the test(s) that cover it.

Selected Test Cases

#	Test Method	Expected	Status	FR
TC-01	Constructor_ZeroCapacity_ThrowsException	ArgumentException on capacity = 0	Pass	FR5/NFR1
TC-02	ReserveSlot_WhenFull_ThrowsException	InvalidOperationException when class full	Pass	FR5
TC-03	BookClass_ActiveMemberAndAvailableClass_ReturnsSuccess	BookingResult.Success = true; booking returned	Pass	FR4
TC-04	BookClass_ExpiredMembership_ReturnsFailure	Success = false, message mentions "expired"; no slot taken	Pass	FR6
TC-05	BookClass_FullClass_ReturnsFailure	Success = false, message mentions "fully booked"	Pass	FR5
TC-06	CancelBooking_ExistingBooking_ReleasesSlot	IsCancelled = true; booked count returns to 0	Pass	FR10
TC-07	CancelBooking_AlreadyCancelled_ReturnsFailure	Second call returns Success = false	Pass	FR10
TC-08	GetMembershipSummary_CountsActiveAndExpired	Correct active/expired counts from mixed list	Pass	FR14
TC-09	GetClassUtilisation_ReturnsBookedAndCapacityPerClass	Booked=2, capacity=5, available=3	Pass	FR15
TC-10	GetTotalCheckIns_CountsAllCheckIns	Returns 2 for a list of 2 check-ins	Pass	FR16
TC-11	Membership blank-ID throws	ArgumentException on empty ID	Pass	FR2
TC-12	CheckIn_ExpiredMembership_ThrowsException	InvalidOperationException on expired member	Pass	FR12

Requirements Traceability Matrix

Req	Summary	Test Evidence
FR1	Register a member (unique ID)	Membership constructor test + web-page duplicate-ID guard
FR2	Reject blank ID/name	TC-11 + blank-name test
FR3	Active/expired based on date	IsActive() tests
FR4	Book when a slot is free	TC-03
FR5	Reject booking when full	TC-05, TC-02
FR6	Reject expired member	TC-04
FR7	Reject past-date booking	Gap - not yet in BookingService

FR8	Prevent booking clashes	Gap - not yet implemented
FR9	Reduce free slots on booking	Slot assertions inside TC-03
FR10	Cancel returns slot back	TC-06, TC-07
FR11	Every booking returns a clear message	Message assertions in TC-03/04/05
FR12	Attendance / reject expired check-in	TC-12
FR13	Only booked members can check in	Gap - CheckIn does not yet link to Booking
FR14	Active vs expired counts	TC-08
FR15	Class utilisation	TC-09
FR16	Attendance numbers	TC-10

Why traceability matters

The RTM does a lot of work for a small table. It shows a marker (and us) where the evidence is for each requirement. It also flags the gaps honestly - FR7, FR8, and FR13 have no test yet because the code doesn't exist yet, and calling that out beats trying to hide it. When we change a requirement later, the RTM tells us exactly which tests need updating. And when a test fails, it tells us which requirement is now at risk. That's what makes it a QA and change-management tool, not just a checklist.

Task 7: Project Progress, Risks and Next Steps

Where we're at

The domain logic for all four modules is built and tested, and the Blazor web app is running. On origin/master we have 30 unit tests, in a shared repository at github.com/kmc4699/ProGym. We've followed the group rules: one repo, both members committing directly to master, no per-person branches. Each of us has averaged more than two meaningful commits a week. The web app has four working pages: Dashboard, Members, Classes, and Bookings; wired up through dependency injection and a shared in-memory data store. We drove every page ourselves end-to-end and confirmed the validation and message rules work through the UI.

Current limitations and risks

- Feature gaps: FR7 (reject past-date bookings), FR8 (booking clash detection), and FR13 (check-in must belong to a real booking) aren't implemented yet. They're small pieces of work but they're honest gaps.
- Cross-module fragility: Booking depends on Membership; Check-in depends on Booking. A change in one can break the other. This became a real issue mid-project, when a past-expiry guard added to Membership's constructor caused five existing tests to fail. We resolved it by introducing an IClock abstraction and a FakeClock test double, which lets tests construct expired memberships without violating the guard. The mitigation going forward is a quick team check whenever a shared class is touched, plus the "master must be green before push" rule.
- In-memory storage: data vanishes when the app restarts. Flagged as out of scope for the prototype but noted as a limitation.

Plan for completing the final phase

- Add FR7, FR8, and FR13 with tests.
- Polish the pages (form resets, better error handling on blank input).
- Fix the duplicate-booking behaviour already flagged as GAP-01, alongside FR8
- Update the RTM to close the current gaps.

Team reflection

Working with two people on one codebase taught us more about process than either of us expected. Committing straight to master with a "green before push" rule sounds strict, but it stops the "works on my branch" problem before it starts. The RTM turned out to be genuinely useful, not just a rubric item - when the past-expiry guard broke tests, we knew immediately which requirements were now uncovered. The pivot from a Windows Forms desktop app to a Blazor web app mid-project was another good lesson: because the domain library was UI-agnostic, none of the tested rules had to change - only the UI project did. That was a big payoff for keeping the layers separate. AI-assisted coding gave us speed on scaffolding and test ideas, but the discipline of running tests immediately (Task 4) is what turned it from risky to useful. Overall, this project has done what an SQA paper is meant to do - it's forced us to think about quality before, during, and after coding, not as an afterthought.

References

International Organization for Standardization. (2011). ISO/IEC 25010:2011 - Systems and software engineering - Systems and software quality requirements and evaluation (SQuaRE) - System and software quality models.

<https://www.iso.org/standard/35733.html>

Microsoft. (n.d.-a). Unit testing C# with MSTest and .NET. Microsoft Learn.

<https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-mstest-intro>

Microsoft. (n.d.-b). ASP.NET Core Blazor. Microsoft Learn.

<https://learn.microsoft.com/en-us/aspnet/core/blazor/>

GitHub. (n.d.). GitHub Copilot documentation.

<https://docs.github.com/en/copilot>

North, D. (2006, March). Introducing BDD. Better Software Magazine.

<https://dannorth.net/introducing-bdd/>